UNIVERSIDADE FEDERAL DO AMAZONAS

Instituto de Computação
ICC205 - Introdução a Bancos de Dados

TRABALHO PRÁTICO II

Armazenamento em Memória Secundária, Hashing e Índices B+Tree

Aluno: Cirus Alexandre Tinga
Professor: Altigran Soares da Silva

Manaus - AM
2026

Sumário

1. Introdução

2. Decisões de Projeto

3. Previsões Antes das Medições

4. Medições com Blocos de 4096 Bytes

5. Medições com Blocos de 8192 Bytes

6. Comparação e Discussão

7. Casos Patológicos

8. Declaração de Uso de Inteligência Artificial

9. Conclusão

1. Introdução

Este trabalho implementa e mede estruturas físicas de armazenamento em memória secundária. O arquivo de dados é organizado por hashing do identificador, e duas B+Trees persistentes fornecem caminhos de acesso por ID e por título. Todas as leituras e escritas dos arquivos de dados e de índice passam pela camada blockio, de forma que a métrica principal seja a quantidade determinística de blocos lógicos acessados.

A mesma base é avaliada com blocos de 4096 e 8192 bytes. As consultas oficiais de queries/oficial.txt são executadas tanto sem índice (findrec, findtitle e findrange) quanto com índice (seek1, seek2 e seekrange).

2. Decisões de Projeto

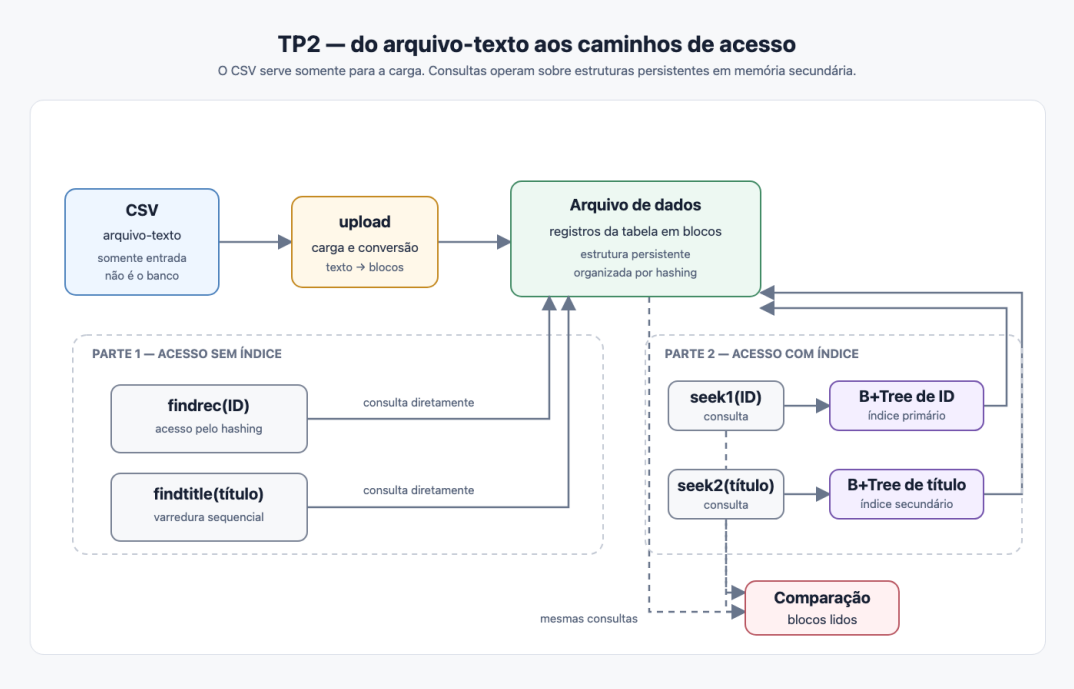


Figura 1 - Fluxo entre CSV, arquivo de dados e índices.

2.1 Representação física dos registros

O registro em memória é representado por Article. Para persistência foi definida a estrutura DiskArticle, de tamanho fixo igual a 1000 bytes. Os campos textuais usam vetores de char com limites fixos: título 301 bytes, autores 151 bytes, atualização 20 bytes e snippet 512 bytes. Textos maiores que a capacidade são truncados. A escolha por tamanho fixo simplifica o cálculo do slot e o acesso por endereço físico (bloco, slot).

2.2 Arquivo hash

Foram definidos 50000 buckets. A função de hashing é h(id) = id mod 50000. Cada bucket começa em um bloco principal; quando ele enche, é criado um bloco de overflow, ligado por um ponteiro next_block. Com 4096 bytes cabem 4 registros por bloco; com 8192 bytes cabem 8.

2.3 B+Tree por ID e por título

A B+Tree por ID armazena nas folhas pares (ID, endereço físico). A B+Tree por título armazena (título, endereço físico) e aceita títulos duplicados. As folhas de ambas são encadeadas. No índice de título, a

busca começa na folha mais à esquerda compatível com a chave e continua por next_leaf enquanto houver entradas iguais.

A capacidade dos nós é calculada em função de BLOCK_SIZE. Considerando a reserva de 64 bytes para metadados/alinhamento, a implementação usa MAX_KEYS = (BLOCK_SIZE - 64) / (tamanho da chave + tamanho do endereço + tamanho do ponteiro).

Índice	4096 - MAX_KEYS	8192 - MAX_KEYS	Altura esperada
ID	144	290	3 níveis em ambos
Título	12	25	5 níveis (4096) e 4 níveis (8192)

3. Previsões Antes das Medições

Antes das medições, a expectativa era que findrec fosse especialmente barato por acessar diretamente o bucket calculado pelo ID, normalmente 1 bloco e mais blocos apenas quando houvesse overflow. seek1 deveria percorrer a B+Tree e então ler um bloco de dados. Como a implementação lê a folha durante find_leaf e novamente durante a inspeção da chave, a previsão prática para seek1 era de aproximadamente 4 leituras no índice mais 1 no arquivo de dados para chaves existentes.

findtitle e findrange necessariamente realizam varredura completa, pois a organização hash é por ID e não preserva ordem nem fornece acesso direto por título. Portanto, a previsão era de 69940 leituras de dados em 4096 e 51134 em 8192, independentemente da quantidade de resultados.

seek2 deveria percorrer uma árvore mais alta que a de ID, devido à chave textual de 301 bytes. Para títulos repetidos, seriam necessárias leituras adicionais de folhas e um acesso ao arquivo de dados para cada ocorrência. seekrange deveria localizar o primeiro ID pela árvore e seguir as folhas encadeadas até ultrapassar o limite final; assim, o custo no índice cresceria com a largura do intervalo e o custo nos dados seria aproximadamente igual ao número de registros retornados, pois não foi implementado cache de blocos de dados entre endereços.

Operação	Previsão principal
findrec	1 bloco na maioria dos buckets; mais em overflow
findtitle	varredura completa do arquivo de dados
findrange	varredura completa do arquivo de dados
seek1	~4 blocos de índice + 1 de dados se existir
seek2	caminho na B+Tree + folhas de duplicatas + 1 dado por ocorrência
seekrange	caminho inicial + folhas sequenciais + 1 dado por resultado

4. Medições com Blocos de 4096 Bytes

A carga com 4096 bytes aceitou 204285 registros e ignorou 2 linhas. O arquivo de dados ficou com 69940 blocos, a B+Tree por ID com 2877 blocos e a B+Tree por título com 27074 blocos. A taxa aproximada de ocupação do arquivo hash foi 71,31%.

Métrica	4096 bytes
Registros por bloco	4
Blocos do arquivo de dados	69940
Blocos do índice ID	2877
Blocos do índice título	27074
Escritas hash	294165
Leituras hash na carga	226422
Escritas índice ID	218451

Leituras índice ID	817800
Escritas índice título	330110
Leituras índice título	1442240

4.1 Consultas oficiais por ID

ID	findrec dados	seek1 índice	seek1 dados	Encontrado
5	1	4	1	sim
316639	1	4	1	sim
663752	1	4	1	sim
1014283	1	4	1	sim
1483294	1	4	1	sim
1	2	4	0	não
2000000	1	4	0	não

Totais: arquivo de dados = 69940 blocos; índice por ID = 2877 blocos.

4.2 Consultas oficiais por título

Título	Resultados	findtitle dados	seek2 índice	seek2 dados
Design and implementation... smartphone platform.	1	69940	8	1
A rotating-magnet... biomedical implants	1	69940	7	1
Review of particle physics particle data group	45	69940	13	45
Two-dimensional gas of massless Dirac fermions in graphene	26	69940	11	26
Um titulo que nao existe na base	0	69940	7	0

Totais: arquivo de dados = 69940 blocos; índice por título = 27074 blocos.

4.3 Consultas oficiais por intervalo

Intervalo	Resultados	findrange dados	seekrange índice	seekrange dados
[1, 50]	10	69940	4	10
[1000, 1100]	20	69940	5	20
[500000, 500500]	99	69940	5	99
[200000, 260000]	3052	69940	46	3052
[1600000, 1700000]	0	69940	4	0

Totais: arquivo de dados = 69940 blocos; índice por ID = 2877 blocos.

5. Medições com Blocos de 8192 Bytes

A carga foi repetida com BLOCK_SIZE=8192. Foram aceitos os mesmos 204285 registros e ignoradas 2 linhas. A capacidade do bloco passou para 8 registros, o arquivo de dados caiu para 51134 blocos, o índice por ID para 1419 blocos e o índice por título para 12416 blocos. A taxa aproximada de ocupação do arquivo hash foi 48,77%.

Métrica	8192 bytes
Registros por bloco	8
Blocos do arquivo de dados	51134
Blocos do índice ID	1419
Blocos do índice título	12416
Escritas hash	256553
Leituras hash na carga	204803
Escritas índice ID	211115

Leituras índice ID	779919
Escritas índice título	261897
Leituras índice título	1120503

5.1 Consultas oficiais por ID

ID	findrec dados	seek1 índice	seek1 dados	Encontrado
5	1	4	1	sim
316639	1	4	1	sim
663752	1	4	1	sim
1014283	1	4	1	sim
1483294	1	4	1	sim
1	1	4	0	não
2000000	1	4	0	não

Totais: arquivo de dados = 51134 blocos; índice por ID = 1419 blocos.

5.2 Consultas oficiais por título

Título	Resultados	findtitle dados	seek2 índice	seek2 dados
Design and implementation... smartphone platform.	1	51134	6	1
A rotating-magnet... biomedical implants	1	51134	6	1
Review of particle physics particle data group	45	51134	9	45
Two-dimensional gas of massless Dirac fermions in graphene	26	51134	7	26
Um titulo que nao existe na base	0	51134	6	0

Totais: arquivo de dados = 51134 blocos; índice por título = 12416 blocos.

5.3 Consultas oficiais por intervalo

Intervalo	Resultados	findrange dados	seekrange índice	seekrange dados
[1, 50]	10	51134	4	10
[1000, 1100]	20	51134	4	20
[500000, 500500]	99	51134	5	99
[200000, 260000]	3052	51134	25	3052
[1600000, 1700000]	0	51134	4	0

Totais: arquivo de dados = 51134 blocos; índice por ID = 1419 blocos.

6. Comparação e Discussão

As medições confirmam que o hashing é muito eficiente para busca pontual por ID. Para os IDs existentes, findrec leu apenas 1 bloco em ambos os tamanhos de página. Em 4096, o ID inexistente 1 exigiu 2 leituras por causa da cadeia de overflow; em 8192, o mesmo bucket coube em um único bloco e a busca caiu para 1 leitura. seek1 manteve 4 leituras de índice em todos os casos e acrescentou 1 leitura de dados quando a chave existia.

Para títulos, a diferença é grande. findtitle precisa ler todo o arquivo para garantir que todas as ocorrências foram encontradas: 69940 blocos em 4096 e 51134 em 8192. Já seek2 acessou de 7 a 13 blocos do índice em 4096 e de 6 a 9 em 8192, além de um bloco de dados para cada resultado. O título "Review of particle physics particle data group" retornou 45 registros; portanto, o custo de dados do seek2 foi 45 blocos nas duas configurações.

Nas consultas por intervalo, findrange também lê todo o arquivo independentemente da cardinalidade. seekrange aumenta seu custo de índice conforme mais folhas são percorridas. O intervalo [200000, 260000], com 3052 resultados, usou 46 leituras de índice em 4096 e 25 em 8192. A redução ocorre porque os nós e folhas comportam mais entradas com páginas maiores.

Estrutura	4096	8192	Variação
Arquivo de dados	69940	51134	-26,9%
Índice ID	2877	1419	-50,7%
Índice título	27074	12416	-54,1%
Registros por bloco	4	8	+100%
MAX_KEYS ID	144	290	+101,4%
MAX_KEYS título	12	25	+108,3%

A redução de blocos de índice confirma que a ordem da B+Tree depende do tamanho do bloco. A árvore por ID permanece com três níveis, mas possui muito menos nós. A árvore por título reduz a altura esperada de cinco para quatro níveis, o que aparece nas buscas: por exemplo, o título único "Design and implementation..." caiu de 8 para 6 leituras no índice.

7. Casos Patológicos

7.1 Chaves inexistentes

Para ID inexistente, findrec ainda precisa examinar o bucket e sua cadeia de overflow. O ID 1 custou 2 blocos em 4096 e 1 em 8192; o ID 2000000 custou 1 em ambos. seek1 concluiu a ausência com 4 leituras de índice e 0 leituras de dados.

Para título inexistente, findtitle sempre realizou varredura completa: 69940 ou 51134 blocos. seek2 concluiu a ausência com 7 leituras de índice em 4096 e 6 em 8192, sem acessar o arquivo de dados.

7.2 Títulos duplicados

O índice secundário mantém uma entrada por ocorrência. Foram encontradas 45 ocorrências de "Review of particle physics particle data group" e 26 de "Two-dimensional gas of massless Dirac fermions in graphene". A busca percorre folhas adjacentes enquanto a chave permanecer igual e depois lê os endereços correspondentes no arquivo de dados.

7.3 Intervalos de cardinalidades diferentes

O custo de findrange não muda com a cardinalidade, porque o arquivo hash precisa ser varrido por completo. No seekrange, a quantidade de blocos do índice cresce com o número de folhas atravessadas: em 4096, os intervalos com 10, 20, 99 e 3052 resultados exigiram respectivamente 4, 5, 5 e 46 leituras no índice. Em 8192, os custos foram 4, 4, 5 e 25.

7.4 Concentração e overflow dos buckets

A partir do arquivo hash produzido, 645 dos 50000 buckets ficaram vazios e a maior concentração observada foi de 13 registros em um único bucket. Com 4096 bytes, 18800 buckets precisaram de overflow; a maior cadeia teve 4 blocos (6 buckets chegaram a esse tamanho). Com 8192 bytes, apenas 1134 buckets precisaram de overflow e a maior cadeia teve 2 blocos. Isso explica parte da redução do custo de algumas buscas hash quando o tamanho do bloco aumenta.

8. Declaração de Uso de Inteligência Artificial

Foi utilizada a ferramenta ChatGPT, da OpenAI, como apoio durante o desenvolvimento. O uso incluiu: esclarecimento do enunciado; revisão da organização dos arquivos; auxílio na depuração de erros de compilação e de execução; discussão da implementação do arquivo hash e das B+Trees; elaboração de comandos de teste; organização das medições de 4096 e 8192 bytes; e revisão textual do relatório.

O código foi executado e validado com a base do trabalho, e as alterações sugeridas foram incorporadas e testadas antes da versão final. Os números apresentados neste relatório são os produzidos pela implementação entregue sobre as consultas oficiais. O estudante permanece responsável por compreender e explicar as decisões de projeto, o código e os resultados na arguição.

9. Conclusão

Os experimentos mostram que a estrutura mais adequada depende do tipo de consulta. Para igualdade por ID, o arquivo hash foi mais barato que a B+Tree na maioria dos casos. Para título e intervalos, os índices B+Tree reduziram drasticamente o número de blocos acessados em comparação com varreduras completas.

A mudança de 4096 para 8192 bytes reduziu o número de blocos do arquivo de dados e, principalmente, dos índices. O benefício foi visível no índice por título e nos intervalos grandes, pois páginas maiores aumentaram a capacidade dos nós e reduziram a quantidade de folhas percorridas. Os resultados também confirmam que índices possuem custo de construção e ocupam espaço adicional, mas oferecem caminhos de acesso muito mais eficientes para consultas que não combinam com a organização hash do arquivo principal.